credentials, Keycloak called Edge Service back and sent the ID Token for the newly authenticated user. Then Edge Service stored the token and redirected the browser to the required endpoint, together with a session cookie. From that point on, any communication between the browser and Edge Service will use that session cookie to identify the authenticated context for that user. No token is exposed to the browser.

The ID Token is stored in `OidcUser`, part of `Authentication` and ultimately included in `SecurityContext`. In chapter 9, we used the Spring Session project to make Edge Service store session data in an external data service (Redis), so it could remain stateless and be able to scale out. `SecurityContext` objects are included in the session data and are therefore stored in Redis automatically, making it possible for Edge Service to scale out without any problem.

Another option for retrieving the currently authenticated user (the *principal*) is from the context associated with a specific HTTP request (called the *exchange*). We'll use that option to update the rate limiter configuration. In chapter 9, we implemented rate-limiting with Spring Cloud Gateway and Redis. Currently the rate-limiting is computed based on the total number of requests received every second. We should update it to apply the rate limits to each user independently.

Open the `RateLimiterConfig` class and configure how the username of the currently authenticated principal should be extracted from the request. If no user is defined (that is, the request is unauthenticated, *anonymous*), we use a default key to apply rate-limiting to all unauthenticated requests as a whole.

Listing 11.10 Configuring rate limiting for each user